

Lab 8

Detailed Design Tasks

Issues

1. Teachers

Issue 1: Architecture Design Diagram Review

Issue 2: Teacher explains the Detailed Design template

2. Students

Task 1: Students prepare Detailed Design document

Issue 1: Architecture Diagram Review

- Feedback:
 - 1.3 Purpose of architecture design: divide the system into subsystems, so they can be developed individually.
 - 2.1 Use case diagram and class diagram
 - Change the **class diagram** into architectural design.
 - Some teams made changes to the class diagram we gave you or even ignored it altogether.
 - Consider **which subsystem** each **use cases** belongs.
 - Many teams didn't follow the given use cases and create their own in the UI.
 - 2.2 Design model includes both MVC and three-layered models

Issue 1: Architecture Diagram Review

- Feedback – 2.3 whole system architecture
 - Whole system architecture diagram is **missing** in some teams.
 - Whole system architecture must be **consistent** with the **class diagram** in 2.1
 - Every class in 2.1 must appear in diagram in 2.3.
 - Every class in 2.3 must appear in diagram in 2.1.
 - The whole system architecture in 2.3
 - Shows how the system is divided into subsystems,
 - Shows how the subsystems are coupled with each other,
 - Contains class names, but **no attributes, operations, UI or database**, those things belong to Section 3 subsections.
 - A class cannot appear in 2 subsystems.
 - Otherwise, those 2 subsystems would be highly coupled together, and cohesion is low with each subsystem.

Issue 1: Architecture Diagram Review

- Feedback – 3.X Subsystems UI
 - Subsystems' UI must be **consistent** with the **use case diagram** in 2.1.
 - All UIs in subsystems must be from the **use case diagram** in 2.1.
 - Except for HomeUI or MainUI, which are links to home pages.
 - **Every use case** in the use case diagram must appear in the **UI layer** of a subsystem.
 - A use case cannot appear in 2 different subsystems.
 - Otherwise, the 2 subsystems are highly coupled together, and cohesion is low with each subsystem.

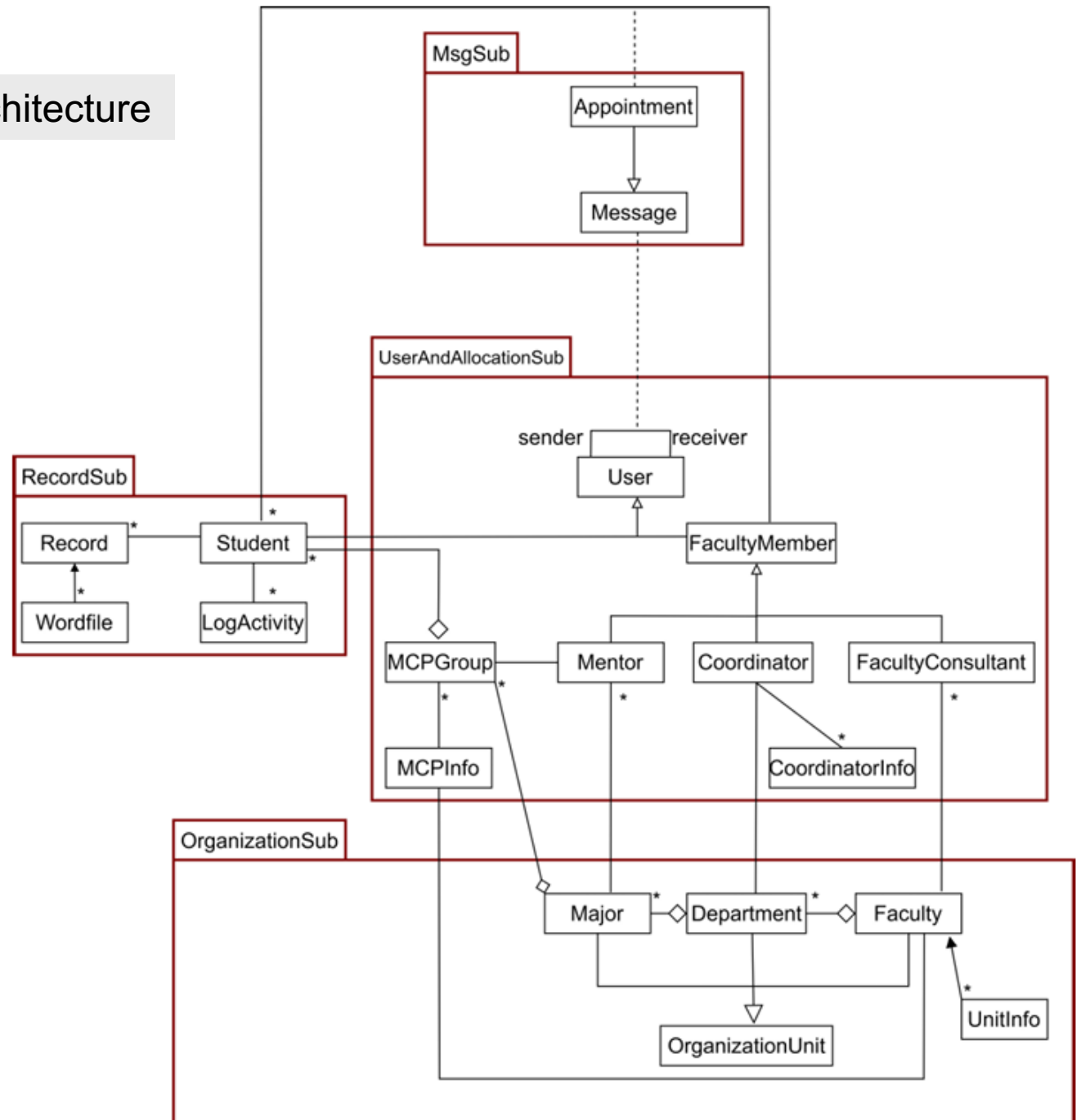
Issue 1: Architecture Diagram Review

- Feedback – 3.X.2 database tables
 - A table's attributes must match the class' attributes.
 - Especially so in the detailed design we do this week.
 - DBMS class thoroughly covered how to use **tables** to **represent associations** between entities.
 - We briefly covered this in SE Lecture 10.
 - Recall the 10 cases in DBMS class; following 3 are most useful.
 - For **1 to many** relation, the **foreign key** from the "**1**" table is "marked for association" in the "**many**" table.
 - For **many to many** relation, the **foreign keys** for **both tables** are "marked for association" in the association table.
 - For **inheritance**, we can **extend the table** to accommodate both the super and sub classes.

Example for struggling teams

- This solution is workable, courtesy of Team C08.
 - Beware, there are small errors in this solution.
 - The feedback for errors are included in the message boxes in the slides.

2.3 Overall System Architecture



3.1 User&AllocationSub

UserAndAllocationInterfaceSub

SearchMentorsInfoUI

CheckMentorInfoUI

UpdateMentorGroupUI

LoginUI

PwdUI

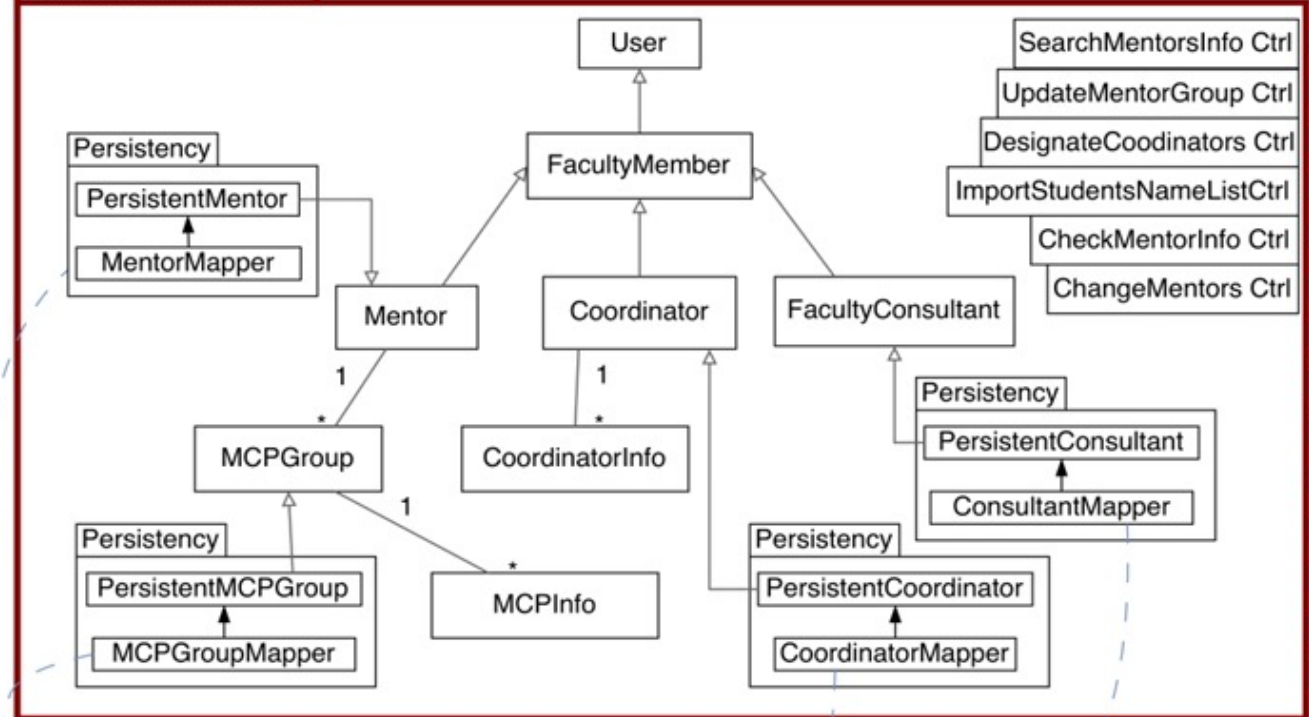
DesignateCoordinatorsUI

ChangeMentorsUI

MainUI

ImportStudentsNameListsUI

UserAndAllocationSub



UserAndAllocationDatabaseSub

MCPGroup, Mentor, Coordinator, FacultyConsultant

3.1.2 Database

Mentor			
MentorID	Name	Email	Office

MCP Group	
GroupID	MentorID(for association)

Coordinator	
CoordinatorID	Name

FacultyConsultant	
FacultyConsultantID	Name

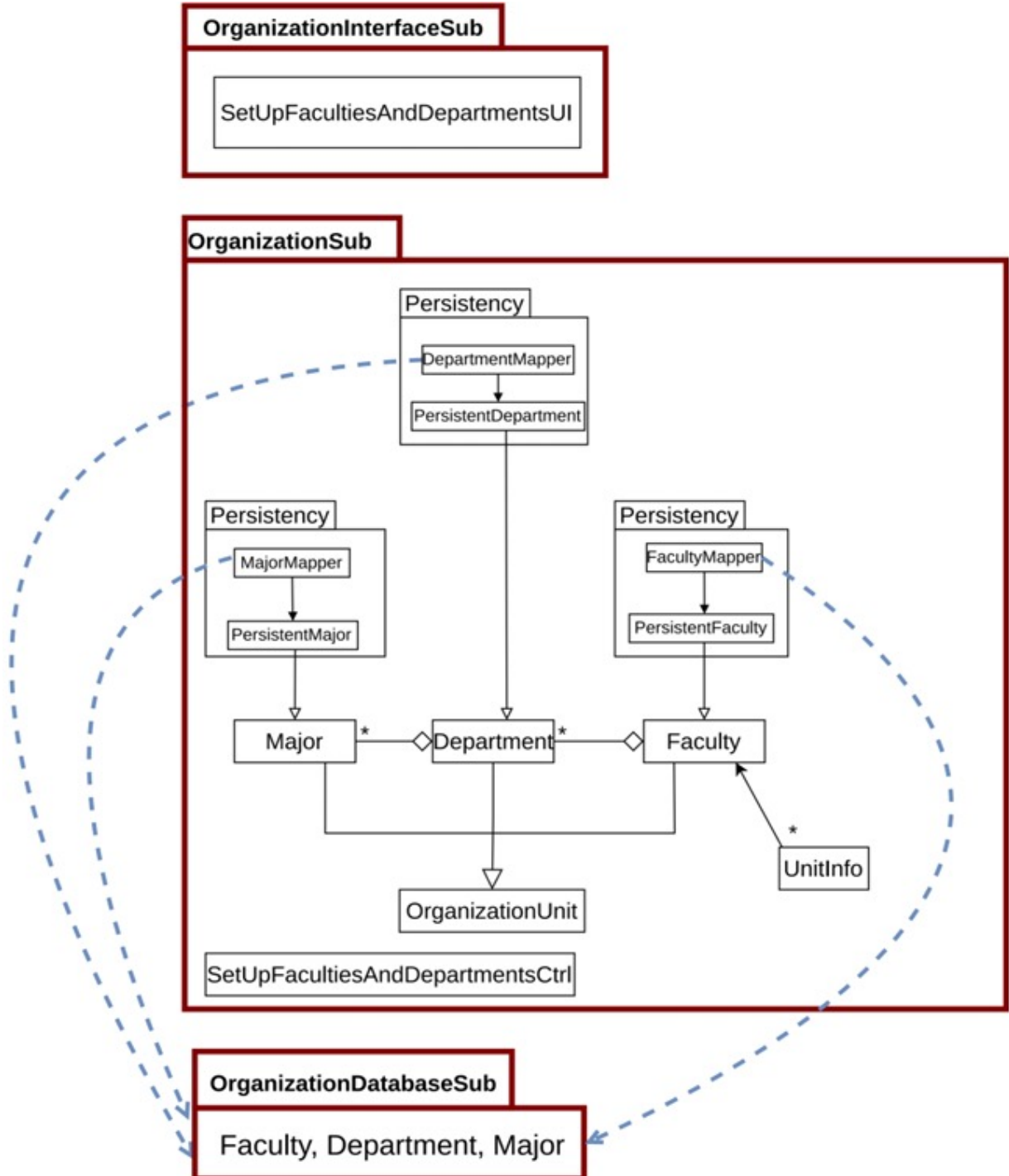
Wrong or missing attributes for associate in a table.

For 1 to many associations, the association should be in the “many” table, with the foreign key as the association. Recall the ten cases in DBMS class.

Mentor needs to have foreign key MajorID as association.

Faculty consultant needs to have foreign key FacultyID as association.

3.2 OrganizationSub



3.2.2 Database

Faculty	
FacultyID	FacultyName

Department		
DepartmentID	DepartmentName	FacultyID (for association)

Major		
MajorID	MajorName	DepartmentID (for association)

3.3 MsgSub

MsgInterfaceSub

CommunicateUI

Controls not consistent with UI's

Better to have CommunicateCtrl
rather than MessageCtrl and
AppointmentCtrl

MsgSub

Persistency

MsgMapper

PersistentMsg

Appointment

Message

Persistency

PersistentAppoint

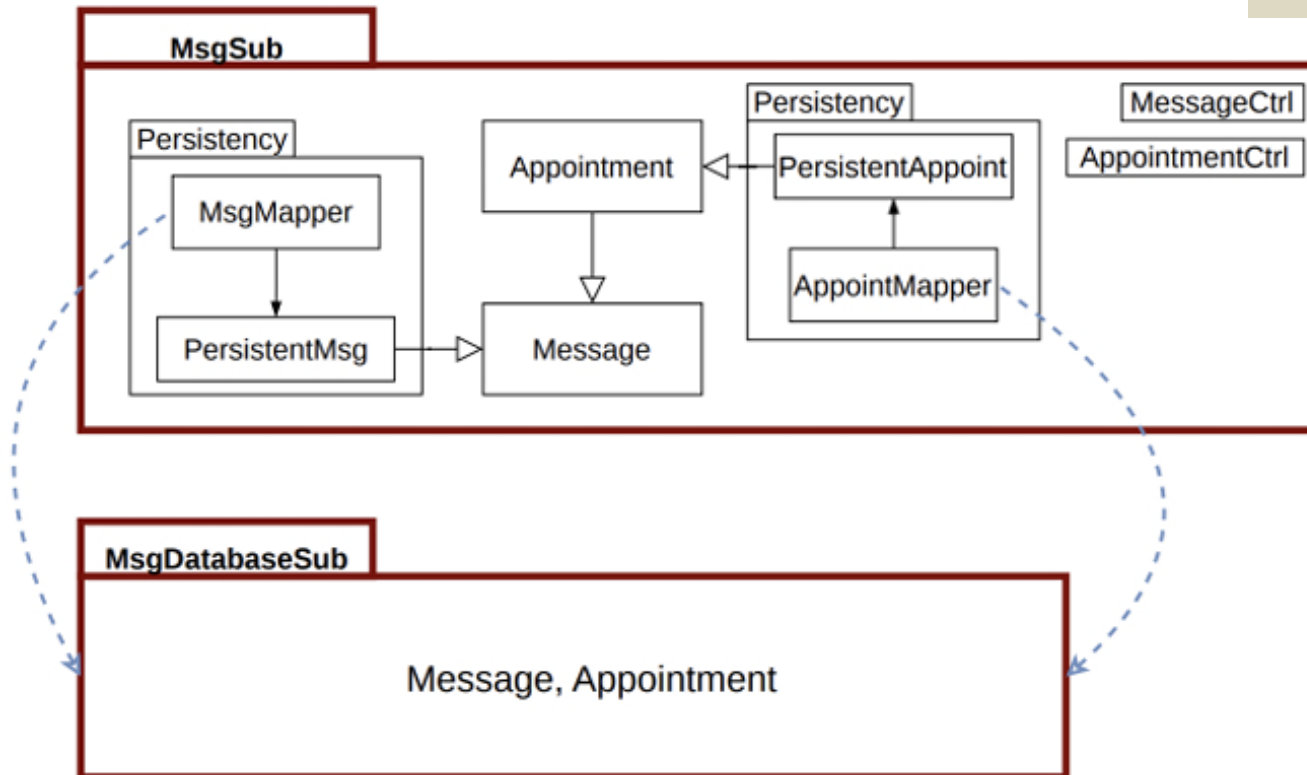
AppointMapper

MessageCtrl

AppointmentCtrl

MsgDatabaseSub

Message, Appointment



3.3.2 Database

Message					
MessageID	Content	Status	Attachment	SenderID (for association)	ReceiverID (for association)

Appointment					
AppointmentID	Time	Date	Venue	StudentID (for association)	MentorID (for association)

RecordInterfaceSub

SearchStudentInfoUI

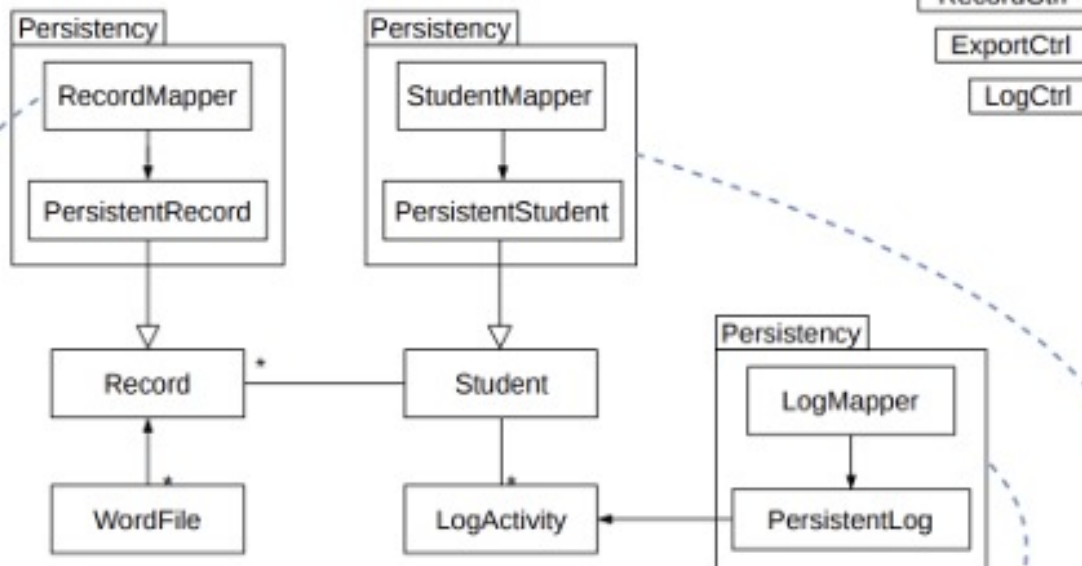
EditInterviewRecordUI

ExportRecordsUI

ViewLogInfoUI

3.4 RecordSub

RecordSub



RecordDatabaseSub

Student, Record, LogActivity

Controls not consistent with UI's
Should have
SearchStudentInfoCtrl etc.

3.4.2 Database

Student	
StudentID	GroupID (for association)

Record					
RecordID	Date	ProblemStatement	InterviewSummary	FollowupAction	StudentID (for association)

LogActivity				
LogID	Time	Data	FunctionAccess	StudentID (for association)

Issue 2: Detailed Design Template Explanation

- Teacher explains the detailed design template. The template is available in iSpace.
- Main points
 - **Section 2.1**: Whole class diagram. Same as the one you put in Section 2.1 in the Architecture Design Document
 - **Section 3.1**: Put the class diagram restructured from Section 2.1
 - Follow the restructuring guidelines in **Lecture 11**, starting around **slide 18**.

Restructuring

- Classes in a subsystem can be further adjusted
 - Change n-ary associations to binary associations
 - Increase the inheritance
 - Collapse (降级) classes with no significant (明显的) behavior into attributes
 - Use qualifier (限定符) to change the one-to-many and many-to-many relationship
 - Implement an association class as a class

Issue 2: Detailed Design Template Explanation (Cont.)

- Starting from 3.2, describe each class in the class diagram
 - Replace 3.#.1 section title with real class name to be described in this section .
 - Put the class notation with all its attributes and operations there before giving any explanation. The class should contain the finished interface design.
 - Following **Lecture 11**, around **slides 7-17**.
 - Explanation is optional, only needed when some new attributes or operations are added from the architecture design.

Issue 2: Detailed Design Template Explanation (Cont.)

- Pre-, post- conditions are usually required for each operation which has pre- and post- conditions in every class.
- However, as a practice in this submission, you only need to **choose one operation from one class** to give pre and post condition.
 - I.e., give pre and post condition to **only ONE operation** in the whole document.
 - Choose an operation that's substantial enough to demonstrate the pre and post conditions; i.e. don't choose an operation that has NONE as the pre and post conditions.
- Remember to **update the Table of Content** after finishing the description for all classes,

Example from last year

- Next slide contains a restructured class diagram for Section 3.1 from a previous year.
- If possible, try to put your entire restructured class diagram in 1 page.
- If that's not practical, break the diagram into 2 pages with the least amount of coupling between pages.
 - The classes near the edge of the two pages must appear on both pages, so we can see how those classes are connected to their neighboring classes.

3.1 Class diagram

The Figure 2 Updated Class Diagram is below:

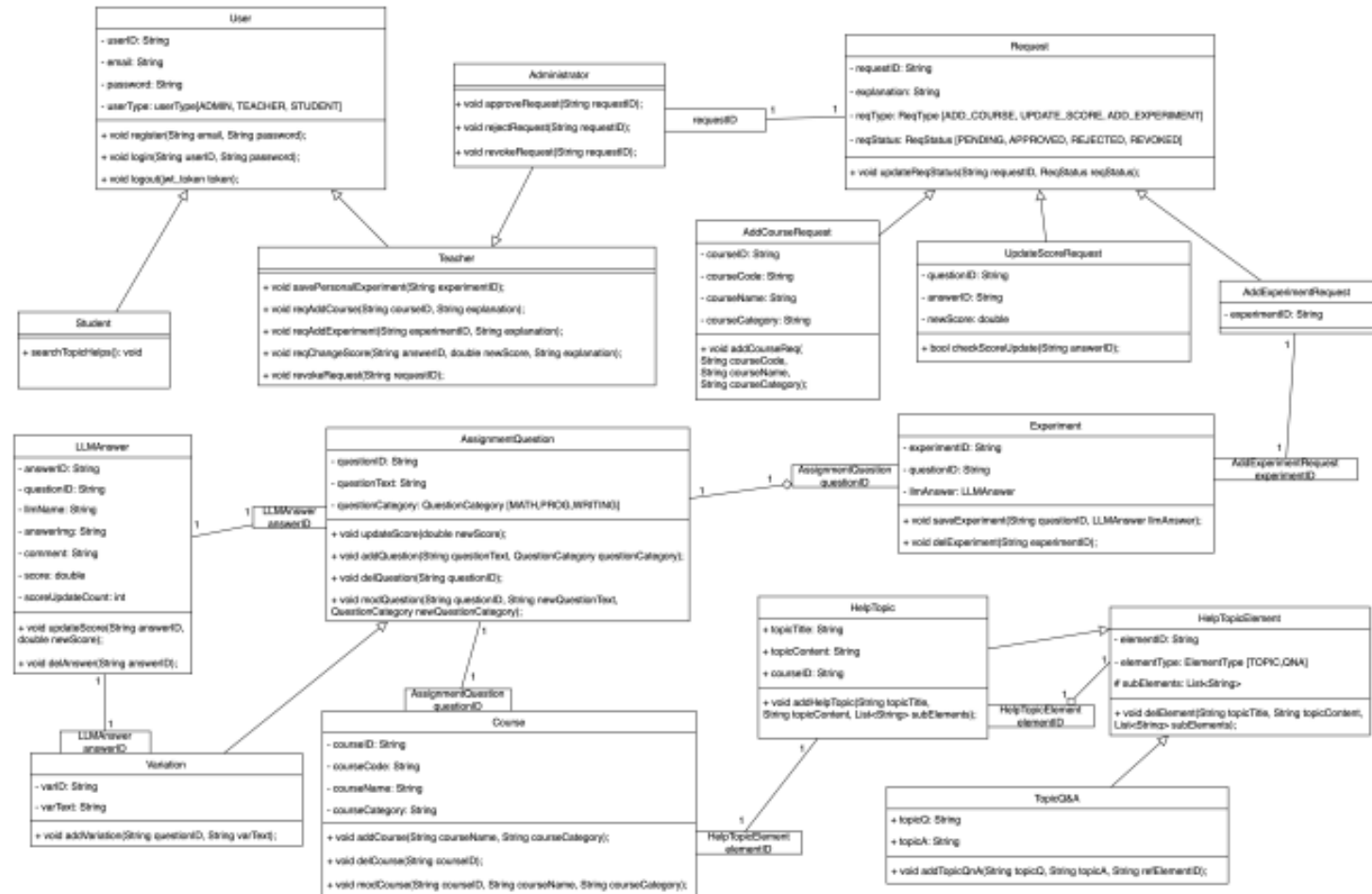


Figure 2 Updated Class Diagram for LLM-Homework

Student Task 1: Prepare Detailed Design Document

- Students finish the Detailed Design Document according to the template.
 - Follow guidelines in “Issue 2: Detailed Design Template Explanation” in previous slides.
 - **Deadline: 23:59 Saturday 25 April 2026**
 - You have 1 more day than usual.
- Each team submits the Detailed Design document, **ONE copy** into iSpace and GitLab.
 - On the cover of the documents next to the student names, leaders (with the support of most of the team) indicate how much each member contributed: Full, Fair, Little or None.

Next Week in Lab 9

- You will watch videos on unit testing.
- Please **bring your earpiece to class**, if you don't like the headphones in the lab.